

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

**Submitted by
P Nithya
(1BF24CS206)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under
VTU) BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by P NITHYA (**1BF24CS206**), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Prof. Shwetha K S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	3
2	Implement Johnson Trotter algorithm to generate permutations.	9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	13
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	17
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	21
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	26 28
9	Implement Fractional Knapsack using Greedy technique.	31
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	35
11	Implement "N-Queens Problem" using Backtracking.	38

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

```
#include <stdio.h>
#include <conio.h>

int a[10][10], indegree[10] = {0}, v[10], stack[10];
int n, k = 0, top = -1;

void readadj();
void topological();

void main()
{
    int i;
    printf(" To find the topological ordering of the vertices\n");

    readadj();
    topological();

    if (k == n)
    {
        printf("\nTopological sequence is:\n");
        for (i = 0; i < n; i++)
        {
            printf("\t%d", v[i]);
        }
    }
    else
    {
        printf("\nTopological sequence does not exist");
    }

    getch();
}

void readadj()
{
    int i, j;

    printf("\nEnter the number of vertices of a graph:\n");
    scanf("%d", &n);

    if (n > 0 && n < 10)
    {
        printf("\nEnter the adjacency matrix:\n");

        for (i = 0; i < n; i++)
```

```

    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    return;
}
else
{
    printf("Enter valid number of vertices\n");
    goto I;
}
}

```

void topological()

```

{
    int i, j, R;

    // Find indegree
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (a[i][j] == 1)
            {
                indegree[j]++;
            }
        }
    }
    for (i = 0; i < n; i++)
    {
        if (indegree[i] == 0)
        {
            stack[++top] = i;
        }
    }
    while (top != -1)
    {
        R = stack[top--];
        v[k++] = R;

        for (i = 0; i < n; i++)
        {
            if (a[R][i] == 1)
            {

```

```
indegree[i]--;  
  
if (indegree[i] == 0)  
{  
    stack[++top] = i;
```

```
}  
}  
}  
}  
}
```

Output:

```
=====
To find the topological ordering of the vertices
=====

Enter the number of vertices of a graph:
6

Enter the adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0

Topological sequence is:
      0      2      4      1      3      5|
```

Lab program 2:

```
#include <stdio.h>
```

```
void printPerm(int a[], int n)
```

```
{
    int i;
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

```
int getMobile(int a[], int dir[], int n)
```

```
{
    int i, mobile = 0;

    for(i = 0; i < n; i++)
    {
        if(dir[i] == -1 && i != 0 && a[i] > a[i - 1])
        {
            if(a[i] > mobile)
                mobile = a[i];
        }

        if(dir[i] == 1 && i != n - 1 && a[i] > a[i + 1])
        {
            if(a[i] > mobile)
                mobile = a[i];
        }
    }

    return mobile;
}
```

```
int main()
```

```
{
    int n, i, j;

    printf("Enter n: ");
    scanf("%d", &n);

    int a[n], dir[n];

    for(i = 0; i < n; i++)
    {
```

```

    a[i] = i + 1;
    dir[i] = -1;    // -1 = LEFT, 1 = RIGHT
}

```

```

printPerm(a, n);

```

```

while(1)
{
    int mobile = getMobile(a, dir, n);

    if(mobile == 0)
        break;

    int pos;
    for(i = 0; i < n; i++)
    {
        if(a[i] == mobile)
        {
            pos = i;
            break;
        }
    }

    if(dir[pos] == -1) // move left
    {
        int temp = a[pos];
        a[pos] = a[pos - 1];
        a[pos - 1] = temp;

        temp = dir[pos];
        dir[pos] = dir[pos - 1];
        dir[pos - 1] = temp;

        pos--;
    }
    else // move right
    {
        int temp = a[pos];
        a[pos] = a[pos + 1];
        a[pos + 1] = temp;

        temp = dir[pos];
        dir[pos] = dir[pos + 1];
        dir[pos + 1] = temp;
    }
}

```

```

        pos++;
    }

    for(i = 0; i < n; i++)
    {
        if(a[i] > mobile)
            dir[i] = -dir[i];
    }

    printPerm(a, n);
}

return 0;
}

```

Output:

```

Enter the Value of n : 4

[ 1] 1234 (4 3)
[ 2] 1243 (3 2)
[ 3] 1423 (2 1)
[ 4] 4123 (4 3)
[ 5] 4132 (1 2)
[ 6] 1432 (2 3)
[ 7] 1342 (3 4)
[ 8] 1324 (2 1)
[ 9] 3124 (4 3)
[10] 3142 (3 2)
[11] 3412 (2 1)
[12] 4312 (4 3)
[13] 4321 (1 2)
[14] 3421 (2 3)
[15] 3241 (3 4)
[16] 3214 (1 2)
[17] 2314 (4 3)
[18] 2341 (3 2)
[19] 2431 (2 1)
[20] 4231 (3 4)
[21] 4213 (1 2)
[22] 2413 (2 3)
[23] 2143 (3 4)
[24] 2134

```

Lab program 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
void merge(int a[], int low, int mid, int high)
```

```
{
    int i = low, j = mid + 1, k = 0;
    int temp[100000];

    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
```

```
    while (i <= mid)
        temp[k++] = a[i++];
```

```
    while (j <= high)
        temp[k++] = a[j++];
```

```
    for (i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];
}
```

```
void mergeSort(int a[], int low, int high)
```

```
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}
```

```
int main()
```

```
{
    int n, i;
    double time_taken;
```

```

clock_t start, end;

printf("Enter number of elements: ");
scanf("%d", &n);

int a[n];

printf("Enter elements:\n");
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

start = clock();

mergeSort(a, 0, n - 1);

end = clock();

printf("\nSorted elements:\n");
for (i = 0; i < n; i++)
    printf("%d ", a[i]);

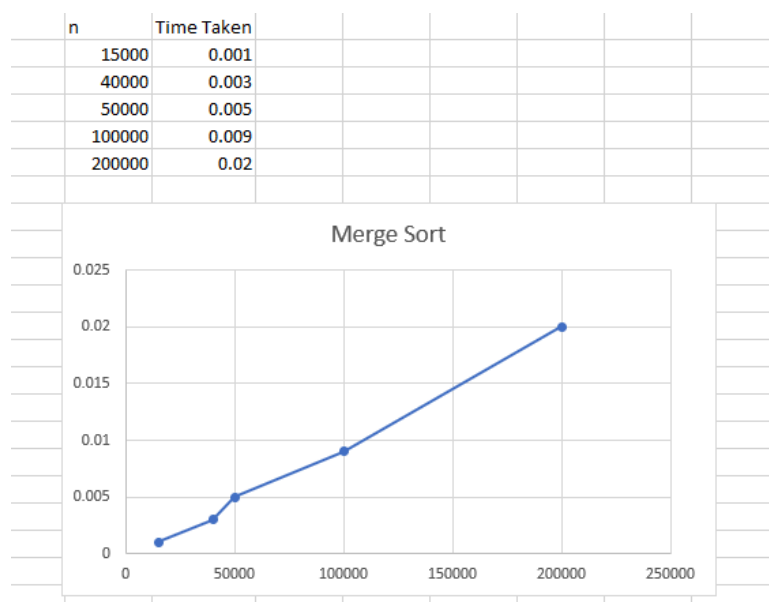
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("\n\nTime taken = %f seconds\n", time_taken);

return 0;
}

```

Output:



```

Enter the number of elements: 50000
Time Taken: 0.005000 seconds

```

Lab program 4:**#include <stdio.h>****#include <stdlib.h>****#include <time.h>****void merge(int a[], int low, int mid, int high)****{****int i = low, j = mid + 1, k = 0;****int temp[100000];****while(i <= mid && j <= high)****{****if(a[i] < a[j])****temp[k++] = a[i++];****else****temp[k++] = a[j++];****}****while(i <= mid)****temp[k++] = a[i++];****while(j <= high)****temp[k++] = a[j++];****for(i = low, k = 0; i <= high; i++, k++)****a[i] = temp[k];****}****void mergeSort(int a[], int low, int high)****{****if(low < high)****{****int mid = (low + high) / 2;****mergeSort(a, low, mid);****mergeSort(a, mid + 1, high);****merge(a, low, mid, high);****}****}****int main()**

```

{
    int n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter value of N: ");
    scanf("%d", &n);

    int a[n];

    srand(time(NULL));

    for(i = 0; i < n; i++)
        a[i] = rand() % 10000;

    start = clock();

    mergeSort(a, 0, n - 1);

    end = clock();

    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Time taken = %f seconds\n", time_taken);

    return 0;
}

```

Output:



```

Enter number of elements: 40000
Time taken: 0.003000 seconds

```

Lab program 5:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void heapify(int a[], int n, int i)
```

```
{
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && a[left] > a[largest])
```

```
        largest = left;
```

```
    if (right < n && a[right] > a[largest])
```

```
        largest = right;
```

```
    if (largest != i)
```

```
    {
```

```
        int temp = a[i];
```

```
        a[i] = a[largest];
```

```
        a[largest] = temp;
```

```
        heapify(a, n, largest);
```

```
    }
```

```
}
```

```
void heapSort(int a[], int n)
```

```
{
```

```
    int i;
```

```
    for (i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(a, n, i);
```

```
    for (i = n - 1; i > 0; i--)
```

```
    {
```

```
        int temp = a[0];
```

```
        a[0] = a[i];
```

```
        a[i] = temp;
```

```
        heapify(a, i, 0);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    int a[1000];
```

```
    clock_t start, end;
```

```
    double time_taken;
```

```
    printf("Enter number of elements: ");
```

```

scanf("%d", &n);

printf("Enter elements:\n");
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

start = clock();

heapSort(a, n);

end = clock();

time_taken = (double)(end - start) / CLOCKS_PER_SEC;

printf("Sorted array:\n");
for (i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\nTime taken = %lf seconds\n", time_taken);

return 0;
}

```

Output:

```

Enter number of elements (max 1024): 6
Enter elements (space separated):
42 10 27 75 90 5

```

```

Sorted elements are:
5 10 27 42 75 90

```

Size	ASC	DESC	RAND
16	163	150	156
32	637	608	632
64	2423	2355	2395
128	9209	9044	9147
256	35330	34984	35182
512	137244	136489	136903

Lab program 6:

```
#include <stdio.h>
```

```
int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}
```

```
int main()
{
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n + 1], val[n + 1];

    printf("Enter weights of items:\n");
    for (int i = 1; i <= n; i++)
    {
        scanf("%d", &wt[i]);
    }

    printf("Enter profits of items:\n");
    for (int i = 1; i <= n; i++)
    {
        scanf("%d", &val[i]);
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    int v[n + 1][W + 1];

    /* Initialize first row */
    for (int w = 0; w <= W; w++)
    {
        v[0][w] = 0;
    }
}
```

```

}

/* Initialize first column */
for (int i = 0; i <= n; i++)
{
    v[i][0] = 0;
}

/* Build DP table */
for (int i = 1; i <= n; i++)
{
    for (int w = 1; w <= W; w++)
    {
        if (wt[i] > w)
        {
            v[i][w] = v[i - 1][w];
        }
        else
        {
            v[i][w] = max(
                v[i - 1][w],
                val[i] + v[i - 1][w - wt[i]]
            );
        }
    }
}

printf("\nDP Table:\n");

for (int i = 0; i <= n; i++)

{
    for (int w = 0; w <= W; w++)
    {
        printf("%4d", v[i][w]);
    }
    printf("\n");
}

printf("\nMaximum Profit = %d\n", v[n][W]);

return 0;

```

}

Output:

```
Enter number of items: 4
Enter weights of items:
2 1 3 2
Enter profits of items:
12 10 20 15
Enter knapsack capacity: 5
```

DP Table:

0	0	0	0	0	0
0	0	12	12	12	12
0	10	12	22	22	22
0	10	12	22	30	32
0	10	15	25	30	37

Maximum Profit = 37

Lab program 7:

```
#include <stdio.h>

#define INF 99999

int main()
{
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int D[n][n];
    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &D[i][j]);

            if (i == j)
            {
                D[i][j] = 0;
            }
            else if (D[i][j] == 0)
            {
                D[i][j] = INF;
            }
        }
    }

    /* Floyd-Warshall Algorithm */
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (D[i][k] != INF &&
```

```

        D[k][j] != INF &&
        D[i][k] + D[k][j] < D[i][j])
    {
        D[i][j] = D[i][k] + D[k][j];
    }
}
}

/* Check for negative cycles */
for (int i = 0; i < n; i++)
{
    if (D[i][i] < 0)
    {
        printf("\nNegative cycle detected!\n");
        return 0;
    }
}
printf("\nShortest distance matrix:\n");
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (D[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", D[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 3 0 7
8 0 2 0
5 0 0 1
2 0 0 0

Shortest distance matrix:
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```

Lab program 8a:

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define MAX 100

int V;

int graph[MAX][MAX];

void printMST(int parent[])
{
    printf("Edge \tWeight\n");

    for (int i = 1; i < V; i++)
    {
        printf("%d - %d \t%d\n",
            parent[i],
            i,
            graph[parent[i]][i]);
    }
}

int minKey(int key[], bool mstSet[])
{
    int min = INT_MAX;
    int min_index;

    for (int v = 0; v < V; v++)
    {
        if (mstSet[v] == false && key[v] < min)
        {
            min = key[v];
            min_index = v;
        }
    }

    return min_index;
}
```

void primMST()

```

{
    int parent[MAX];
    int key[MAX]; bool
    mstSet[MAX];

    for (int i = 0; i < V; i++)
    {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++)
    {
        int u = minKey(key, mstSet);

        mstSet[u] = true;

        for (int v = 0; v < V; v++)
        {
            if (graph[u][v] &&
                mstSet[v] == false &&
                graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printMST(parent);
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter the adjacency matrix:\n");

```

```

for (int i = 0; i < V; i++)
{
    for (int j = 0; j < V; j++)
    {
        scanf("%d", &graph[i][j]);
    }
}

primMST();

return 0;
}

```

Output:

```

Enter number of vertices: 5
Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5

```

Lab program 8b:

```
#include <stdio.h>
#include <stdlib.h>

int comparator(const void *a, const void *b)
{
    return ((int *)a)[2] - ((int *)b)[2];
}

void makeSet(int parent[], int rank[], int n)
{
    for (int i = 0; i < n; i++)
    {
        parent[i] = i;
        rank[i] = 0;
    }
}

int findParent(int parent[], int component)
{
    if (parent[component] == component)
        return component;

    return parent[component] =
        findParent(parent, parent[component]);
}

void unionSet(int u, int v, int parent[], int rank[])
{
    if (rank[u] < rank[v])
    {
        parent[u] = v;
    }
    else if (rank[u] > rank[v])
    {
        parent[v] = u;
    }
    else
    {
        parent[v] = u;
        rank[u]++;
    }
}
```

```

    }
}

void kruskalAlgo(int V, int E, int edge[E][3])
{
    qsort(edge, E, sizeof(edge[0]), comparator);

    int parent[V], rank[V];

    makeSet(parent, rank, V);

    int minCost = 0;
    int count = 0;
    printf("Edges in MST:\n");

    for (int i = 0; i < E; i++)

    {
        int u = edge[i][0];
        int v = edge[i][1];
        int wt = edge[i][2];

        int p1 = findParent(parent, u);
        int p2 = findParent(parent, v);

        if (p1 != p2)
        {
            unionSet(p1, p2, parent, rank);
            printf("%d - %d : %d\n", u, v, wt);
            minCost += wt;
            count++;

            if (count == V - 1)
                break;
        }
    }

    printf("Total cost: %d\n", minCost);
}

```

```

int main()
{
    int V, E;

    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);

    int edge[E][3];
    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < E; i++)
    {
        printf("Edge %d: ", i + 1);
        scanf("%d %d %d",
            &edge[i][0],
            &edge[i][1],
            &edge[i][2]);
    }

    kruskalAlgo(V, E, edge);

    return 0;
}

```

Output:

```

Enter number of vertices and edges: 4 5
Enter edges (u v weight):
Edge 1: 0 1 10
Edge 2: 1 3 15
Edge 3: 2 3 4
Edge 4: 2 0 6
Edge 5: 0 3 5
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total cost: 19

```

Lab program 9:

```
#include <stdio.h>
```

```
struct Item
```

```
{
```

```
    int profit;
```

```
    int weight;
```

```
    float ratio;
```

```
};
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    float capacity, totalProfit = 0.0;
```

```
    printf("Enter the number of items: ");
```

```
    scanf("%d", &n);
```

```
    struct Item item[n], temp;
```

```
    printf("Enter profit and weight of each item:\n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("Item %d\n", i + 1);
```

```
        printf("Profit: ");
```

```
        scanf("%d", &item[i].profit);
```

```
        printf("Weight: ");
```

```
        scanf("%d", &item[i].weight);
```

```
        item[i].ratio =
```

```
            (float)item[i].profit / item[i].weight;
```

```
    }
```

```
    printf("Enter the capacity of knapsack: ");
```

```
    scanf("%f", &capacity);
```

```
    /* Sort items in descending order of ratio */
```

```
    for (i = 0; i < n - 1; i++)
```

```

{
    for (j = i + 1; j < n; j++)
    {
        if (item[i].ratio < item[j].ratio)
        {
            temp = item[i];
            item[i] = item[j];
            item[j] = temp;
        }
    }
}

printf("\nSelected Items:\n");

for (i = 0; i < n; i++)

{
    if (capacity >= item[i].weight)
    {
        printf(
            "Item with Profit=%d Weight=%d taken completely\n",
            item[i].profit,
            item[i].weight);

        totalProfit += item[i].profit;
        capacity -= item[i].weight;
    }
    else
    {
        float fraction =
            capacity / item[i].weight;

        printf(
            "Item with Profit=%d Weight=%d taken %.2f fraction\n",
            item[i].profit,
            item[i].weight,
            fraction);

        totalProfit += item[i].profit * fraction;
        capacity = 0;
        break;
    }
}

```

}

```
printf("\nMaximum Profit = %.2f\n", totalProfit);

return 0;
}
```

Output:

```
Enter the number of items: 3
Enter profit and weight of each item:
Item 1
Profit: 60
Weight: 10
Item 2
Profit: 100
Weight: 20
Item 3
Profit: 120
Weight: 30
Enter the capacity of knapsack: 50

Selected Items:
Item with Profit=60 Weight=10 taken completely
Item with Profit=100 Weight=20 taken completely
Item with Profit=120 Weight=30 taken 0.67 fraction

Maximum Profit = 240.00
```

Lab program 10:

```
#include <stdio.h>
#include <limits.h>

int d[10], p[10], visited[10];

void dijkstra(int a[10][10], int s, int n)
{
    int u, v, i, j, min;

    for (v = 0; v < n; v++)
    {
        d[v] = INT_MAX;
        p[v] = -1;
        visited[v] = 0;
    }

    d[s] = 0;

    for (i = 0; i < n; i++)
    {
        min = INT_MAX;
        u = -1;

        for (j = 0; j < n; j++)
        {
            if (visited[j] == 0 && d[j] < min)
            {
                min = d[j];
                u = j;
            }
        }

        if (u == -1)
            break;

        visited[u] = 1;

        for (v = 0; v < n; v++)
        {
            if (a[u][v] > 0 &&
```

```

        visited[v] == 0 &&
        d[u] != INT_MAX &&
        d[u] + a[u][v] < d[v])
    {
        d[v] = d[u] + a[u][v];
        p[v] = u;
    }
}
}
}

```

```

void path(int v, int s)
{
    if (p[v] != -1)
        path(p[v], s);

    if (v != s)
        printf("->%d", v);
}

```

```

void display(int s, int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (i != s)
        {
            printf("%d", s);
            path(i, s);

            if (d[i] == INT_MAX)
                printf(" = INF\n");
            else
                printf(" = %d\n", d[i]);
        }
    }
}

```

```

int main()
{
    int a[10][10];

```

```

int i, j, n, s;
printf("Program for Single Source Shortest Path using Dijkstra's Algorithm\n\n");

printf("Enter the number of vertices (size of the graph): ");

scanf("%d", &n);
printf("Enter the %d x %d cost matrix:\n", n, n);

for (i = 0; i < n; i++)

{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &a[i][j]);
    }
}

printf("Enter the source vertex: ");
scanf("%d", &s);
dijkstra(a, s, n);

printf("\nOUTPUT\n");

printf("The shortest paths from %d to remaining vertices are:\n\n", s);

display(s, n);

printf("\nThank You\n");

return 0;
}

```

Output:

```

Program for Single Source Shortest Path using Dijkstra's Algorithm

Enter the number of vertices (size of the graph): 5
Enter the 5 x 5 cost matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter the source vertex: 0

OUTPUT
The shortest paths from 0 to remaining vertices are:

0->1 = 10
0->3->2 = 50
0->3 = 30
0->3->2->4 = 60

```

Lab program 11:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
```

```
int b[20], count = 0;
```

```
int place(int row, int col)
{
    int i;

    for (i = 1; i < row; i++)
    {
        if (b[i] == col || abs(b[i] - col) == abs(row - i))
        {
            return 0;
        }
    }

    return 1;
}
```

```
void dis(int n)
{
    int i, j;
    printf("\nSolution number: %d\n", ++count);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            if (b[i] == j)
            {
                printf("\tQ");
            }
            else
            {
                printf("\t*");
            }
        }
        printf("\n");
    }

    printf("\n\n");
}
```

```
void queen(int row, int n)  
{  
    int col;
```

```

    for (col = 1; col <= n; col++)
    {
        if (place(row, col))
        {
            b[row] = col;

            if (row == n)
            {
                dis(n);
            }
            else
            {
                queen(row + 1, n);
            }
        }
    }
}

void main()
{
    int n;

    printf("=====\n");
    printf("\tN - Queens Problem\n");
    printf("=====\n");

```

I:

```

    printf("\nEnter the number of queens to be placed on an n*n chessboard:\n");
    scanf("%d", &n);

    if (n > 0 && n < 20)
    {
        queen(1, n);

        printf("\nTotal number of solutions: %d\n", count);
    }
    else
    {
        printf("Enter a valid number of queens\n");
        goto I;
    }

    getch();
}

```

Output:

```
=====
      N - Queens Problem
=====

Enter the number of queens to be placed on an n*n chessboard:
4

Solution number: 1
  *      Q      *      *
  *      *      *      Q
  Q      *      *      *
  *      *      Q      *

Solution number: 2
  *      *      Q      *
  Q      *      *      *
  *      *      *      Q
  *      Q      *      *

Total number of solutions: 2
```